

**UNIVERSIDAD DE CARTAGENA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SOFTWARE**

**ANÁLISIS Y APLICACIÓN DE LOS FUNDAMENTOS DE LA
INGENIERÍA DE SOFTWARE Y LA ARQUITECTURA DE
COMPUTACIÓN**

CASO DE ESTUDIO TECNOLÓGICO: RAPPI

Estudiante: Esteban David Marrugo Jassir

Código: 7502620036

Docente: Jhon Carlos Arrieta Arrieta

Asignatura: Introducción a la Ingeniería de Software

Semestre: 2026-2

Cartagena de Indias, D. T. y C., Colombia

Septiembre de 2026

Sustentación Oral en Video

<https://drive.google.com/file/d/1r6U9CzhQhf1wT5PHsAOBYXstoveyKyOY/view?usp=sharing>

2. Introducción

El software ya no es un simple conjunto de programas aislados ejecutándose en una computadora personal. En nuestra sociedad hiperconectada, sostiene cadenas logísticas complejas, intercambios económicos inmediatos y la interacción humana en tiempo real. Esta actividad tiene como propósito analizar de manera crítica y aplicada los fundamentos teóricos de la ingeniería de software y la arquitectura computacional, conectando conceptos abstractos —como concurrencia, gestión de memoria, ciclos de instrucción y protocolos de comunicación— con el funcionamiento de una plataforma tecnológica de escala masiva.

Para aterrizar este estudio, seleccioné como caso de análisis a **Rappi**, la plataforma multilatinas de comercio rápido (*quick-commerce*) y entrega a domicilio nacida en Colombia en 2015. Analizar Rappi resulta especialmente enriquecedor para un estudiante de ingeniería de software porque no se trata de una aplicación única. Es un ecosistema distribuido donde interactúan tres aplicaciones móviles distintas (el usuario final que compra, el repartidor que transporta y el comercio que prepara los pedidos), coordinadas por una infraestructura en la nube que procesa miles de eventos por segundo, calcula rutas geográficas dinámicas y procesa transacciones financieras con tolerancia a fallos.

3. Objetivos de Aprendizaje

A partir del estudio del módulo y la formulación de esta actividad práctica, definí los siguientes objetivos de aprendizaje:

- 1. Analizar la descomposición modular y los requerimientos no funcionales** de un sistema distribuido de alta demanda, identificando cómo los principios de ingeniería de software mitigan la crisis del software y garantizan la escalabilidad.
- 2. Examinar el modelo de negocio y las dinámicas de la industria del software** aplicadas al comercio bajo demanda (*on-demand delivery*), evaluando el impacto del desarrollo continuo y las arquitecturas orientadas a eventos.
- 3. Comprender la interacción física entre hardware y software**, analizando el rol de la jerarquía de memoria (caché, RAM, almacenamiento secundario) y la capacidad de cómputo multinúcleo tanto en dispositivos móviles heterogéneos como en servidores de nube.

4. **Evaluar el papel de los periféricos y sensores de entrada contemporáneos** (módulos GPS, pantallas táctiles capacitivas, cámaras digitales y lectores biométricos) en la recolección fidedigna de datos para la toma de decisiones algorítmicas.
5. **Contrastar la evolución histórica de los sistemas computacionales en las últimas tres décadas**, reconociendo los hitos tecnológicos que hicieron viable la computación ubicua frente a las limitaciones de los sistemas legados.

4. Desarrollo de la Actividad

4.1. Descripción del Sistema Seleccionado: Rappi

Rappi es un ecosistema tecnológico hiperlocal que opera bajo el modelo de plataforma multilateral (*multi-sided platform*). Su propósito principal consiste en conectar la demanda espontánea de bienes de consumo diario (alimentos de restaurantes, compras de supermercado, medicamentos y servicios de mensajería urgente) con la oferta de comercios aliados y una flota distribuida de repartidores independientes.

A nivel de arquitectura de software, Rappi funciona mediante una **red tripartita** coordinada por un núcleo transaccional en la nube:

1. **Aplicación del Consumidor (Consumer App):** Orientada a la experiencia de usuario (*UX/UI*), búsqueda de productos mediante catálogos con latencia ultrabaja, seguimiento en vivo de pedidos sobre mapas vectoriales y procesamiento de pagos electrónicos.
2. **Aplicación del Repartidor (Courier App / Rappitendero):** Diseñada para la recepción de solicitudes mediante subastas algorítmicas, navegación punto a punto mediante geocercas (*geofencing*) y telemetría continua de ubicación física.
3. **Portal y Terminal del Comercio (Merchant Portal):** Interfaz web o tablet dedicada a la gestión de inventario en tiempo real, confirmación de órdenes de cocina, tiempos estimados de preparación y métricas de venta.
4. **Backend Central y Motores de Asignación:** Conjunto de microservicios encargados de resolver el problema de optimización de rutas (enrutamiento de vehículos), balanceo de carga, facturación y detección de fraudes en milisegundos.

4.2. Análisis de Ingeniería de Software

A. *Qué problema resuelve el sistema*

Rappi ataca un problema fundamental de optimización logística y fricción temporal: la asincronía entre la necesidad inmediata de un bien por parte del consumidor y la disponibilidad física de ese producto en una ciudad densamente poblada.

Antes de este tipo de plataformas, un restaurante debía contratar mensajeros de planta fija, arriesgando costos ociosos en horas valle y colapso de entregas en horas pico. Rappi resuelve esto agregando demanda y tercerizando la capacidad de transporte elástica, reduciendo los tiempos de espera promedio a menos de 35 minutos y permitiendo entregas ultrarrápidas de conveniencia (*Rappi Turbo*) en menos de 10 minutos mediante almacenes de proximidad (*dark stores*).

B. *Tipo de software*

Rappi no entra en una sola categoría; es un **sistema de software heterogéneo y altamente distribuido**:

- **Software móvil para clientes y repartidores:** Aplicaciones nativas e híbridas compiladas para los sistemas operativos Android e iOS.
- **Software web empresarial (SaaS):** Paneles administrativos para aliados comerciales y equipos de analítica interna desarrollados sobre tecnologías web modernas.
- **Backend transaccional y orientado a eventos:** Cientos de microservicios comunicados a través de APIs RESTful, gRPC y corredores de mensajería como Apache Kafka.
- **Sistemas de tiempo real:** Conexiones persistentes bidireccionales mediante WebSockets para streaming de coordenadas y estados de preparación.

C. Principios de ingeniería de software necesarios para su desarrollo

Construir una plataforma de esta escala sin principios sólidos de ingeniería garantiza el colapso operativo. Entre los principios esenciales destacan:

1. Alta cohesión y bajo acoplamiento (Microservicios):

El servicio de cobro con tarjeta no puede estar acoplado al servicio de renderizado del menú de hamburguesas. Si la pasarela de pagos experimenta latencia, el usuario aún debe poder navegar el catálogo. Cada microservicio encapsula una única responsabilidad de negocio (*Bounded Context* de Domain-Driven Design).

2. Tolerancia a fallos y resiliencia (*Graceful Degradation*):

Uso de patrones de diseño como *Circuit Breaker* (interruptor de circuito) y reintentos exponenciales con fluctuación (*exponential backoff with jitter*). Si el servicio de recomendaciones personalizadas por inteligencia artificial falla, el sistema degrada con elegancia y muestra una lista estática de platos populares en lugar de arrojar un error 500 en la pantalla del usuario.

3. Consistencia eventual vs. transaccionalidad estricta (Teorema CAP):

En un sistema distribuido con miles de escrituras concurrentes, la disponibilidad y la tolerancia a particiones priman sobre la consistencia inmediata en vistas de catálogo. Sin embargo, para la asignación de dinero y el débito en billeteras digitales, el sistema aplica transacciones ACID estrictas para evitar el problema del doble gasto.

4. Seguridad y privacidad por diseño:

Cifrado de datos en reposo y en tránsito (TLS 1.3), tokenización de credenciales bancarias conforme al estándar PCI-DSS y anonimización de números telefónicos entre clientes y repartidores mediante servidores proxy de voz y mensajería.

D. Equipo de desarrollo necesario

Un sistema de esta magnitud requiere una estructura organizativa interdisciplinaria dividida en células de producto (*squads* multifuncionales):

- **Ingenieros de Frontend Móvil (Android/iOS):** Especializados en ciclo de vida móvil, consumo eficiente de batería, renderizado de mapas por GPU y manejo de estados locales.

- **Ingenieros de Backend y Sistemas Distribuidos:** Encargados de diseñar microservicios robustos, diseñar esquemas de bases de datos relacionales y NoSQL, y gestionar la comunicación asíncrona de eventos.
- **Científicos de Datos e Ingenieros de Machine Learning:** Diseñan los modelos predictivos para estimar tiempos de cocina (*prep-time prediction*), optimizar las tarifas dinámicas y agrupar múltiples pedidos en una sola ruta de entrega (*batching*).
- **Ingenieros de Confiabilidad del Sitio (SRE) y DevOps:** Gestionan la infraestructura en la nube, clústeres de orquestación de contenedores (Kubernetes), pipelines de integración y entrega continua (CI/CD) y monitoreo de métricas operativas (Prometheus, Grafana).
- **Diseñadores de Producto (UI/UX) y Product Managers:** Traducen las fricciones del usuario en especificaciones técnicas claras, asegurando que un repartidor en moto pueda interactuar con la interfaz con un solo toque y sin distracciones peligrosas.
- **Ingenieros de Calidad y Pruebas (QA / Test Automation):** Crean suites de pruebas automatizadas unitarias, de integración y de carga para asegurar que una actualización no rompa el checkout en pleno fin de semana.

4.3. Análisis de la Industria del Software

A. Sector de la industria en que se ubica

Rappi se sitúa en la convergencia de varios sectores de la economía digital:

- **Comercio rápido y logística bajo demanda (*Quick-Commerce / On-Demand Delivery*):** Intermediación logística digital.
- **Economía de plataformas (*Gig Economy*):** Conexión de oferta y demanda laboral flexible mediante algoritmos de asignación.
- **Tecnología financiera (*Fintech*):** A través de sus módulos de billetera digital, procesamiento de cobros y emisión de tarjetas de crédito/débito vinculadas.

B. Modelo de negocio

Rappi no vende comida propia en la mayoría de sus verticales; comercializa intermediación y capacidad tecnológica. Su modelo de ingresos se compone de múltiples flujos:

1. **Comisiones por transacción a comercios aliados:** Cobro porcentual (habitualmente entre el 15% y el 30%) sobre el valor bruto de cada pedido realizado a través de la vitrina digital.

2. **Tarifas de servicio y costo de entrega cobradas al consumidor:** Cobro por el uso de la plataforma tecnológica y el desplazamiento logístico, que varía según distancia, clima y demanda.
3. **Modelo de suscripción recurrente (*RappiPrime / Prime Plus*):** Pago mensual o anual que otorga a los usuarios envíos gratuitos ilimitados, ofertas exclusivas y acceso a servicios complementarios, garantizando un flujo de ingresos predecible (*MRR - Monthly Recurring Revenue*).
4. **Publicidad digital dentro de la aplicación (*Retail Media Ads*):** Marcas de consumo masivo y cadenas de restaurantes pagan para posicionar sus productos en los primeros lugares de búsqueda o en banners patrocinados.
5. **Servicios financieros (Fintech B2B y B2C):** Intereses y comisiones por procesamiento de pagos con tarjetas asociadas y préstamos de capital de trabajo a restaurantes aliados.

C. Tendencias tecnológicas observadas en su desarrollo

- **Computación en la nube elástica y arquitectura sin servidor (*Serverless*):** Capacidad de autoescalar miles de contenedores en cuestión de minutos durante eventos como partidos de fútbol o promociones masivas, apagando recursos sobrantes a medianoche para recortar gastos de infraestructura.
- **Uso intensivo de algoritmos de optimización combinatoria e Inteligencia Artificial:** Modelos matemáticos que resuelven variantes del problema del viajante de comercio (*Travelling Salesperson Problem*) para asignar el repartidor idóneo considerando dirección del tráfico, clima y pedidos en paralelo.
- **Adopción de arquitecturas *Event-Driven*:** Uso de registros distribuidos en tiempo real para desacoplar el flujo de órdenes; la creación de un pedido dispara eventos independientes para cocina, cobro, asignación y auditoría contable.

4.4. Explicación de la Arquitectura de Computación

A. Dispositivos en que se ejecuta el sistema

El ecosistema de Rappi se ejecuta sobre un entorno de hardware profundamente heterogéneo:

- **Lado del cliente y repartidor:** Teléfonos inteligentes de gama baja, media y alta con arquitecturas basadas en procesadores ARM (SoC como Qualcomm Snapdragon, MediaTek Dimensity o Apple Silicon).
- **Lado del comercio:** Tabletas comerciales Android/iPadOS y computadores de escritorio estándar x86_64 para puntos de venta (POS).
- **Lado del servidor e infraestructura central:** Servidores blade de alta densidad en centros de datos de proveedores de nube pública (como Amazon Web Services o Google Cloud Platform), con procesadores multinúcleo x86_64 (Intel Xeon, AMD EPYC) y procesadores ARM para servidores (AWS Graviton), acompañados de aceleradores de inferencia por hardware (GPU/TPU) para modelos predictivos.

B. Papel del procesador y la memoria en su funcionamiento

1. En el dispositivo móvil (Terminal del usuario y repartidor)

- **El Procesador (CPU y GPU):**

La CPU ejecuta el hilo principal de la interfaz de usuario a 60 o 120 cuadros por segundo, calcula la lógica de renderizado y decodifica las respuestas en formato JSON que llegan desde la red.

La GPU (*Graphics Processing Unit*) asume la carga matemática pesada: dibuja los vectores del mapa cartográfico, calcula transformaciones geométricas para rotar la brújula y suaviza las animaciones de la ruta del repartidor en movimiento.

Asimismo, los núcleos de bajo consumo del procesador ARM gestionan los hilos en segundo plano para escuchar eventos del sistema operativo sin agotar la batería en media hora.

- **La Memoria (RAM y Caché):**

La memoria RAM almacena la pila de vistas activas, las imágenes del catálogo en resoluciones adaptadas para evitar que el recolector de basura (*Garbage Collector*) congele la pantalla, y los sockets de red abiertos.

La memoria caché del procesador (L1, L2, L3) es decisiva para que los bucles de cálculo de distancias euclidianas o geográficas entre coordenadas latitud/longitud se resuelvan en nanosegundos sin acudir al bus de memoria principal.

Por su parte, el almacenamiento Flash (UFS/eMMC) actúa como memoria secundaria persistente mediante bases de datos embebidas (SQLite/Room/Realm) para permitir que la aplicación abra al instante y conserve el carrito de compras si el usuario pierde la señal celular en un ascensor.

2. En la infraestructura de servidores (Backend en la nube)

- **El Procesador (CPU del servidor):**

Maneja la concurrencia masiva. Mediante modelos multihilo y programación asíncrona no bloqueante (*Event Loops*), procesa decenas de miles de solicitudes HTTP/gRPC por segundo. La CPU calcula las matrices de costo de viaje y ejecuta las funciones de cifrado criptográfico para cada pago seguro.

- **La Memoria (RAM y Caching Distribuido):**

La memoria RAM en los servidores es el recurso más crítico. En lugar de consultar discos duros o unidades de estado sólido cada vez que alguien busca una hamburguesa, el sistema utiliza clústeres de memoria en red como **Redis** y **Memcached**.

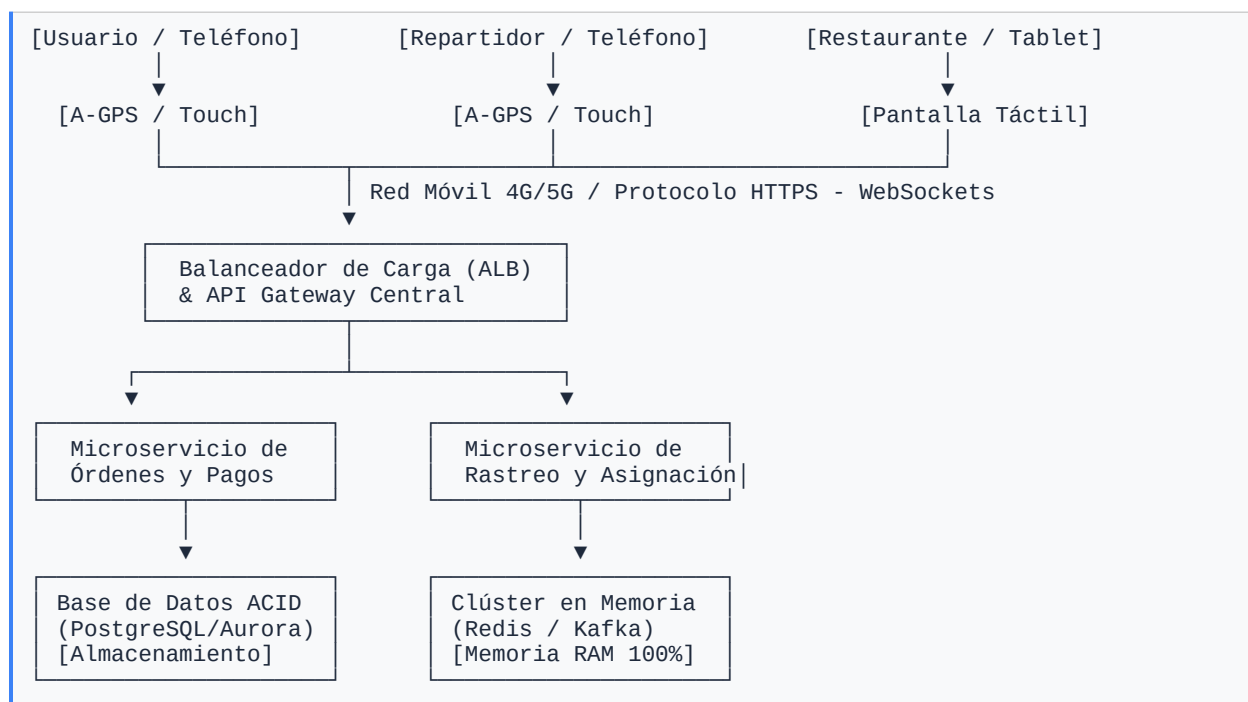
Las sesiones activas, los catálogos calientes y la ubicación en vivo de miles de repartidores se mantienen 100% en memoria RAM. Dado que el acceso a RAM toma decenas de nanosegundos frente a los milisegundos que toma leer un disco SSD, el caching en memoria es la única razón por la cual la plataforma no colapsa en horas de alta demanda.

C. Arquitectura computacional de soporte

Rappi no se hospeda en una máquina física propia; se soporta sobre una **arquitectura de computación en la nube distribuida y elástica**:

- **Nube híbrida y multizona:** Despliegue en múltiples zonas de disponibilidad para garantizar que si un centro de datos físico sufre un corte de energía o fibra óptica, el tráfico se redirija automáticamente a otra zona sin interrupción del servicio.
- **Virtualización y Contenedores:** Aislamiento de microservicios dentro de contenedores Docker gestionados por Kubernetes, permitiendo que un fallo de memoria en el microservicio de búsqueda no afecte al microservicio de autenticación.

- **Malla de Servicios (*Service Mesh*)**: Controla el tráfico interno entre microservicios, inyectando seguridad con cifrado mutuo (mTLS) y balanceo de carga inteligente.



4.5. Análisis de Periféricos y Sistemas de Entrada

A. Dispositivos de entrada que utiliza el sistema

Para que el software tome decisiones correctas, depende de la captura precisa de datos del mundo exterior a través de periféricos y sensores especializados:

1. **Módulo GNSS / GPS Asistido (A-GPS)**: Es el periférico de entrada más importante. Recibe señales de satélites en órbita y torres de telefonía celular para convertir ondas electromagnéticas en coordenadas vectoriales (latitud, longitud, altitud y velocidad).
2. **Pantalla táctil capacitiva (*Touchscreen*)**: Panel con una cuadrícula de sensores que detectan los cambios en la capacitancia eléctrica producidos por el dedo humano, permitiendo registrar toques, gestos de arrastre (*scroll*) y pellizcos (*pinch-to-zoom*) en el mapa.
3. **Cámara digital con sensor CMOS**: Utilizada para capturar fotografías de prueba de entrega (verificación del pedido en portería), escanear códigos de barras de productos en supermercados y validar la identidad del repartidor mediante biometría facial.

4. **Sensores inerciales (Acelerómetro y Giroscopio):** Miden aceleraciones lineales y cambios en la orientación espacial del teléfono. Esto permite al algoritmo de la aplicación del repartidor deducir si el usuario va en moto, bicicleta o a pie, y si el teléfono está montado en un soporte vehicular.
5. **Micrófono:** Permite la entrada de audio para notas de voz directas entre cliente y repartidor al detallar instrucciones de acceso ("el timbre blanco junto a la reja negra").
6. **Sensores biométricos (Lector de huella dactilar y sensores faciales 3D):** Autentican transacciones financieras de manera instantánea mediante el enclave seguro del procesador (*Secure Enclave / Trusted Execution Environment*).

B. Periféricos que permiten interactuar con él (Salida y retroalimentación)

- **Pantalla OLED/IPS de alta densidad:** Traduce mapas, listas de productos y tiempos de entrega en estímulos visuales con contraste dinámico.
- **Motor de vibración háptica (*Haptic Engine*):** Proporciona retroalimentación táctil al repartidor cuando entra una nueva orden o al usuario cuando el domiciliario está a menos de 100 metros, alertándolo sin obligarlo a mirar la pantalla.
- **Altavoces y transductores acústicos:** Emiten alertas sonoras distintivas que destacan sobre el ruido ambiental de una cocina o del tráfico callejero.
- **Impresoras térmicas de tickets (en restaurantes aliados):** Periférico de salida conectado por Bluetooth o red local que imprime automáticamente la comanda en papel térmico para los cocineros al confirmarse la orden.

C. Cómo el usuario introduce información en el sistema

El flujo de entrada de información combina métodos directos e indirectos:

- **Entrada directa (consciente):** El usuario teclea en el teclado virtual en pantalla para buscar platos específicos, selecciona botones de opciones (tipo de término de cocción, bebidas), ajusta la propina y confirma el pedido.
- **Entrada contextual e indirecta (automatizada):** El sistema captura datos sin que el usuario tenga que escribirlos: la geolocalización actual por GPS al abrir la app, la intensidad de la señal de red, el porcentaje de batería restante del teléfono y los patrones de navegación histórica. Esta combinación alimenta el motor de recomendaciones en tiempo real.

4.6. Reflexión sobre Evolución Tecnológica

A. Cómo habría sido un sistema similar hace 20 o 30 años

Hace 25 o 30 años (década de 1990 o principios de los 2000), un sistema con la inmediatez y alcance de Rappi era **técnicamente inviable**. La entrega a domicilio existía, pero dependía de una infraestructura completamente analógica y fragmentada:

- **Canal de entrada:** Llamada por teléfono de línea fija con marcación por tonos o disco rotatorio.
- **Registro y gestión:** La persona del restaurante tomaba la orden a mano con bolígrafo sobre una libreta de papel autocopiativo. Si la línea estaba ocupada, el cliente debía colgar y volver a intentar varias veces.
- **Navegación y periféricos:** El domiciliario era un empleado contratado por el local, quien utilizaba guías cartográficas impresas en papel (como la Guía Roji o planos urbanos en papel) para memorizar la ruta antes de salir. No existía forma de saber dónde estaba el mensajero una vez cerraba la puerta.
- **Transacción económica:** Exclusivamente en efectivo al llegar a la puerta, exigiendo que el domiciliario llevara monedas y billetes para dar el cambio manual.

B. Cambios tecnológicos que permitieron su desarrollo actual

La existencia de Rappi hoy no es casualidad; es el fruto directo de la convergencia de varios saltos tecnológicos ocurridos en las últimas tres décadas:

1. **La masificación del Smartphone con A-GPS integrado:** La llegada del iPhone en 2007 y el florecimiento del ecosistema Android transformaron un teléfono móvil en una computadora de bolsillo de alto rendimiento equipada con sensores de posición global de bajo costo y conexión constante.
2. **El despliegue de redes móviles de alta velocidad y baja latencia (3G, 4G LTE y 5G):** El salto de las lentas conexiones WAP/GPRS al estándar LTE permitió transmitir datos enriquecidos (imágenes en alta definición de platos, streaming continuo de coordenadas cada 3 segundos) con paquetes de datos accesibles.
3. **El advenimiento de la computación en la nube elástica (*Cloud Computing*):** AWS, GCP y Azure eliminaron la necesidad de comprar y mantener servidores físicos costosos en

cuartos fríos. Las empresas nacientes pasaron de pagar grandes sumas en capital inicial (*CapEx*) a pagar únicamente por los recursos de cómputo que consumen por segundo (*OpEx*).

4. **La consolidación de pasarelas de pago digitales y banca abierta:** La estandarización de APIs bancarias seguras y billeteras móviles permitió confiar el débito automático en segundos, mitigando el riesgo de robo de efectivo en calle.
5. **Avances en algoritmos de sistemas distribuidos y bases de datos NoSQL:** Motores capaces de manejar millones de lecturas y escrituras por segundo en memoria RAM sin bloqueos de tablas tradicionales.

C. Tendencias que influirán en su evolución futura

El sector del comercio rápido seguirá mutando en la próxima década impulsado por nuevas fronteras de la ingeniería de software y el hardware:

- **Entregas autónomas mediante robótica terrestre y drones aéreos:** Reemplazo progresivo del transporte tripulado en rutas de corta distancia mediante vehículos autónomos equipados con sensores LiDAR, visión por computador e inferencia de redes neuronales directamente en el dispositivo (*Edge AI*).
- **Logística predictiva anticipatoria:** Algoritmos de aprendizaje profundo que no esperarán a que el cliente ordene; basándose en patrones históricos, clima y horarios, prepararán o despacharán productos hacia almacenes satélite antes de que se confirme la compra.
- **Interfaces conversacionales y multimodales con agentes de IA:** Transición de interfaces basadas en listas y botones hacia asistentes inteligentes capaces de entender comandos de voz naturales complejos ("*Rappi, pídemelo el almuerzo de siempre pero cámbiame la bebida por agua con gas y que llegue en media hora*").
- **Computación de borde (*Edge Computing*):** Procesamiento de modelos de optimización de rutas directamente en terminales locales para reducir el consumo de ancho de banda y garantizar operación sin latencia incluso ante pérdidas intermitentes de conectividad.

5. Conclusiones

El análisis de Rappi como caso de estudio evidencia que el software de impacto contemporáneo nunca opera en el vacío; existe gracias a una simbiosis perfecta con el hardware que lo soporta y los principios de ingeniería que lo ordenan.

Desde la perspectiva de la ingeniería de software, observamos que sin modularidad, desacoplamiento en microservicios y patrones de tolerancia a fallos, la concurrencia masiva colapsaría la plataforma en sus momentos de mayor rentabilidad. En cuanto a la industria, Rappi refleja la madurez de la economía digital en Latinoamérica, transformando el transporte informal en una red logística organizada por algoritmos y monetizada a través de múltiples flujos diversificados.

Por último, el estudio de la arquitectura computacional y los sistemas de entrada nos demuestra la relevancia de la jerarquía de memoria y los periféricos de estado sólido: la sincronización entre el sensor GPS en el bolsillo de un domiciliario en moto, los registros de un procesador ARM y los clústeres de memoria RAM en un centro de datos en la nube es lo que hace posible que una orden de comida llegue a nuestra puerta en minutos. Como futuros ingenieros de software, nuestra tarea no consiste únicamente en escribir código funcional, sino en diseñar con rigor arquitecturas seguras, eficientes y humanamente responsables que soporten el funcionamiento de la sociedad moderna.

6. Referencias Bibliográficas

- Comer, D. E. (2015). *Redes de computadoras e Internet* (6.^a ed.). Pearson Educación.
- Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2.^a ed.). Addison-Wesley Professional.
- Hennessy, J. L., & Patterson, D. A. (2019). *Computer Architecture: A Quantitative Approach* (6.^a ed.). Morgan Kaufmann.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

- Patterson, D. A., & Hennessy, J. L. (2018). *Computer Organization and Design: The Hardware/Software Interface* (RISC-V Edition). Morgan Kaufmann.
- Pressman, R. S., & Maxim, B. R. (2020). *Ingeniería del software: un enfoque práctico* (9.^a ed.). McGraw-Hill Interamericana.
- Sommerville, I. (2011). *Ingeniería de software* (9.^a ed.). Pearson Educación.
- Stallings, W. (2018). *Organización y arquitectura de computadores* (10.^a ed.). Pearson Educación.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4.^a ed.). Pearson.